



DSA Roadmap

Beginner to Pro

Your Path to Mastering Data Structures and Algorithms

About the Speaker



Abhinav Awasthi

- Software Developer at **Zeta - Directi**
- **4+ years** of teaching experience
- A 5-star rated coder on **CodeChef** and an Expert on **Codeforce**



What is the most valuable thing we consider today?



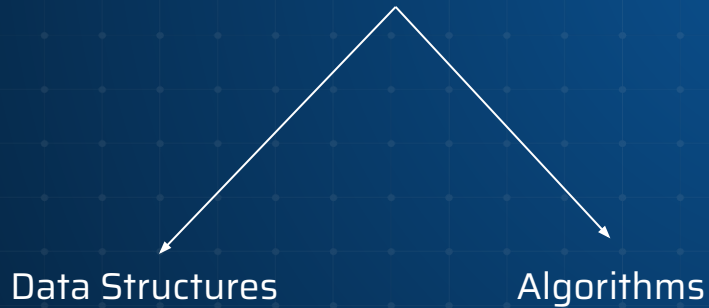
Other than time, what else do we consider immensely valuable?



What is DSA?

Before we start the session, let's discuss DSA in a broad perspective.

**DSA forms the very fundamental of
computer science**



DATA STRUCTURES



Data structures are used to store and organize data in a way that allows for efficient access and manipulation of the data.



ALGORITHMS



Algorithms are the procedures and techniques used to manipulate and process the data in the data structures.



But why?

Why DSA important?



Importance of DSA

- **Efficiency:** DSA provides optimized ways of organizing, manipulating, and retrieving data.
- **Reusability:** Once a data structure or algorithm is implemented, it can be applied to various scenarios, saving time and effort in future projects.
- **Problem-solving:** DSA provides a structured and systematic approach to problem-solving.
- **Industry-standard:** DSA is a fundamental skill that is expected from computer science and software engineering professionals.



Importance of DSA



800 Million Users

Who searching for a user is this fast?

Linear Search



Binary Search



Importance of DSA

Google Maps



How it finds shortest distance?



How to start with DSA?

DSA is -

20% learning
80% practice



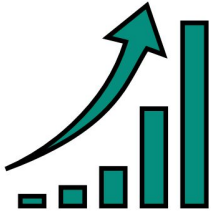
How to start with DSA?

HOW DSA?

1 Consistency is the key!!

2 It's more about practicing

CONSISTENCY



PRACTICE



IMPLEMENT



COMPETE



Let's have a walkthrough to the Roadmap

1. **Foundation Stage**
2. **Core Concepts**
3. **Linear DSA and Algorithms**
4. **Non Linear DSA**
5. **Advanced Algorithms**
6. **Practice, Practice & Practice**



Foundation Stage

- **Programming languages:** Python, C++, Java (choose one)
- **Key concepts:** Variables, Loops, Conditional Statements, Functions
- Input and Output



Core Concepts

- Understanding Time Complexity and Space Complexity
- Recursion
- Pattern Printing



Linear Data Structure

- Arrays & Strings
- Linked Lists
- Stacks & Queues
- Hash Maps & Hash Sets



Algorithms - Level 1

- Sorting and Searching (Binary Search, QuickSort, MergeSort)
- Two Pointer Technique
- Sliding Window Approach
- Greedy Algorithms
- Backtracking



Non Linear Data Structure

- Tree
- Graph
- Heaps
- Shortest Path Algorithms



Advanced Concepts

- Dynamic Programming (Knapsack, LCS, Matrix Chain Multiplication)
- Advanced Trees (Segment Tree, Fenwick Tree)
- String Algorithms (KMP, Rabin-Karp, Z-algorithm)

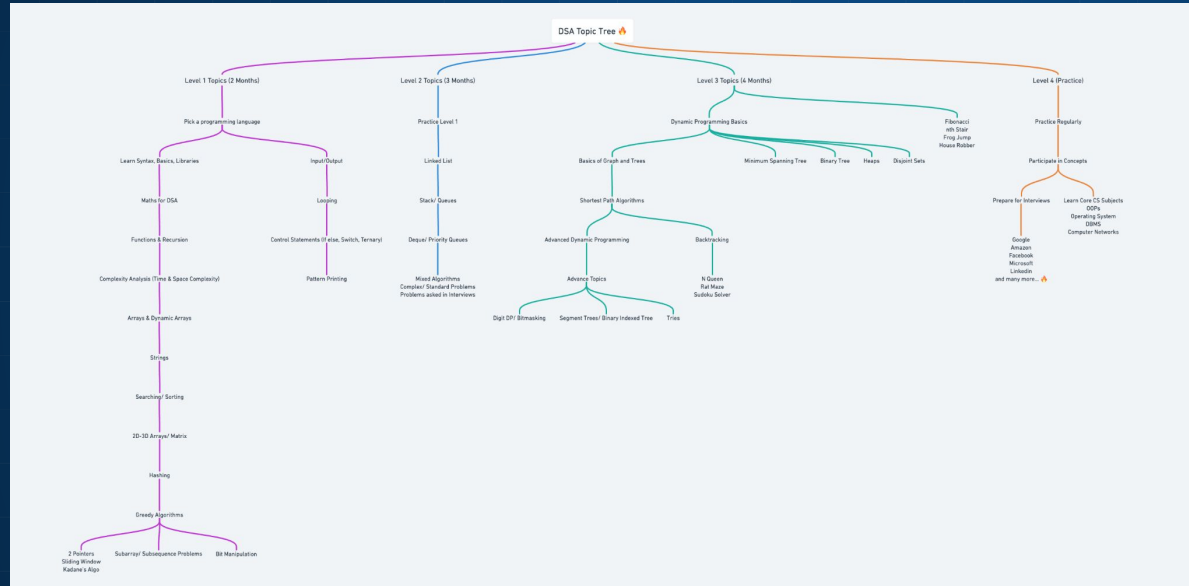


Practice & Platforms

- Start with easy problems, focus on learning patterns.
- Gradually increase difficulty.
- Participate in contests to build speed and accuracy.



Practice & Platforms





**Believe. You're halfway there. Keep
hustling to the finish line.**

Thanks!